Introduction to Integer Linear Programming

BUS 104: Decision Analysis and Management Science

Danko Turcic

October 20, 2026

UNIVERSITY OF CALIFORNIA
UC RIVERSIDE School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Course Information

- **Instructor:** Danko Turcic, Associate Professor of Operations Management
- **Quarter:** Fall 2025
- **Office:** Anderson-1 421
- **Meeting Time:** MW 11:00 AM - 12:20 PM
- **Email:** danko.turcic@ucr.edu
- **Classroom:** SB 165
- **Website:** https://eLearn.ucr.edu
- **Office Hours:** MW 10:00 - 10:50 AM

What is Integer Linear Programming?

Linear Program (LP)

- Optimization problem where the objective function and constraints are linear.
- All decision variables (x_i) are **continuous** (real) values.
- Solution methods (e.g., Simplex) are highly efficient in practice.

Integer Linear Program (ILP)

- An LP with the added restriction that some/all variables are **integers**.
- If all variables are integers, it is a *pure ILP*.
- Ideal for discrete choices (yes/no, counts).
- Generally $\mathcal{NP}$-hard.

Key Difference: Variable Domain Restriction ($\mathbb{R}$ vs. $\mathbb{Z}$)

A Simple Pure ILP Formulation

Consider a two-variable problem (e.g., producing two products x_1, x_2) subject to resources.

Objective Function

Maximize profit:

$$\max z = 3x_1 + 2x_2$$

Constraints

$$\begin{aligned} (C_1) \quad & 2x_1 + x_2 \leq 7 \\ (C_2) \quad & -x_1 + x_2 \leq 3 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Integer Requirement (ILP)

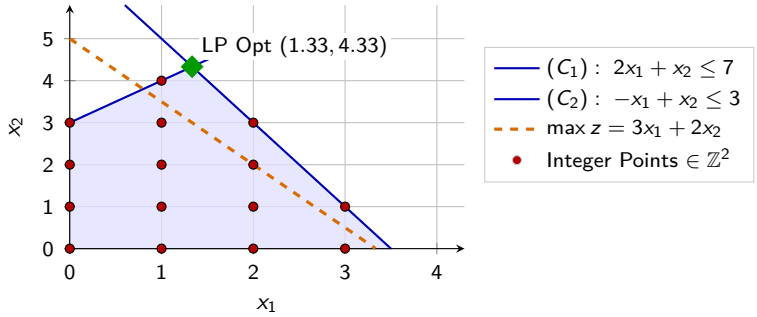
All variables must be integers:

$$x_1, x_2 \in \mathbb{Z}_{\geq 0}$$

LP Relaxation

First solve assuming continuous $x_1, x_2 \in \mathbb{R}_{\geq 0}$.

Visualizing the Feasible Space

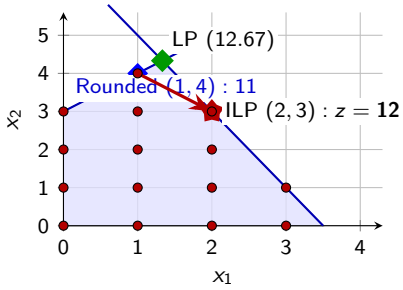


- The shaded region is the continuous LP feasible space; red dots (•) are the 15 integer-feasible solutions.
- **LP Optimum (◊):** $(x_1^*, x_2^*) = (4/3, 13/3)$ with $z^* = 12.67$.

Why the LP Solution is Not Enough

The common mistake is to simply round the fractional LP solution to the nearest integer point.

- **LP Optimum:** $(4/3, 13/3) \approx (1.33, 4.33)$, $z \approx 12.67$.
- **Nearest Integer Points (Rounding):**
  - Down-round: $(1, 4)$, $z = 3(1) + 2(4) = \mathbf{11}$ (feasible).
  - Up-round: $(2, 5)$ (infeasible: $2(2) + 5 = 9 \not\leq 7$).
  - Mixed: $(2, 4)$ (infeasible: $2(2) + 4 = 8 \not\leq 7$).



Rounding can be feasible but sub-optimal.

True ILP Optimum:

- Search the integer lattice points.
- Optimum at $\star$: $(2, 3)$, $z = 3(2) + 2(3) = \mathbf{12}$.

Conclusion: The true ILP optimum ($\star$) beats the rounded LP point, requiring systematic ILP algorithms.

Solving the Pure ILP in Python (PuLP 4.0)

Example 1: Continuous LP Relaxation vs. Pure ILP

```python
import pulp

# 1. Initialize Pure ILP Model (PuLP 4.0 API)
prob = pulp.LpProblem("Simple_Pure_ILP", pulp.LpMaximize)

# 2. Integer Decision Variables: x1, x2 in Z_{>= 0}
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpInteger)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpInteger)

# 3. Objective Function
prob += 3 * x1 + 2 * x2, "Total_Profit"

# 4. Resource Constraints
prob += 2 * x1 + x2 <= 7, "C1"
prob += -x1 + x2 <= 3, "C2"

# 5. Solve and inspect Result
res = prob.solve()
print(f"Status: {res.status.name}")
print(f"Optimal z* = {pulp.value(prob.objective):.2f}")
print(f"x1* = {x1.varValue}, x2* = {x2.varValue}")
```

Output Comparison

- **LP Relaxation (`LpContinuous`):**
  - $x_1^* = 1.33,\ x_2^* = 4.33$
  - $z_{\text{LP}}^* = 12.67$
- **Pure ILP (`LpInteger`):**
  - `Status: Optimal`
  - $x_1^* = 2.0,\ x_2^* = 3.0$
  - $z_{\text{ILP}}^* = \mathbf{12.00}$

PuLP 4.0 Tip

Use `prob.add_variable(..., cat=pulp.LpInteger)` to enforce integrality and `res = prob.solve()` to capture the solution status object.

Section 1

Branch and Bound Method

Branch-and-Bound (B&B): The Idea

Why B&B?

- LP relaxation is fast but may yield fractional values.
- ILP requires integrality; naive rounding can be suboptimal or infeasible.

Core Mechanics

1. **Relax** integrality and solve the LP $\Rightarrow$ upper bound (UB).
2. **Branch** on a fractional variable ($x_i \leq \lfloor x_i^* \rfloor$ vs. $x_i \geq \lceil x_i^* \rceil$).
3. **Bound:** prune subproblems whose LP bound $\leq$ best known integer solution (incumbent), or infeasible.
4. Continue until all nodes are pruned or integer.

Goal: Prove optimality by excluding all non-promising regions.

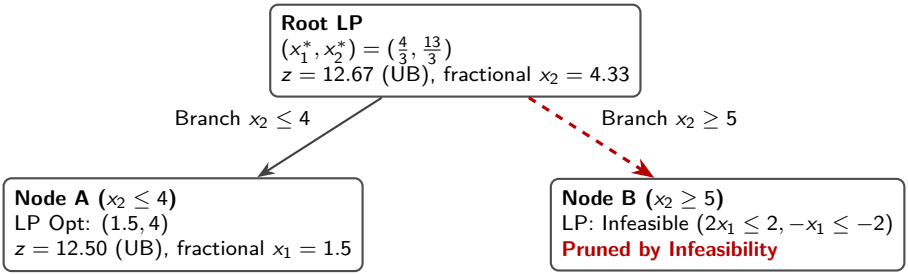
Our ILP (Same as Before)

$$\max z = 3x_1 + 2x_2$$

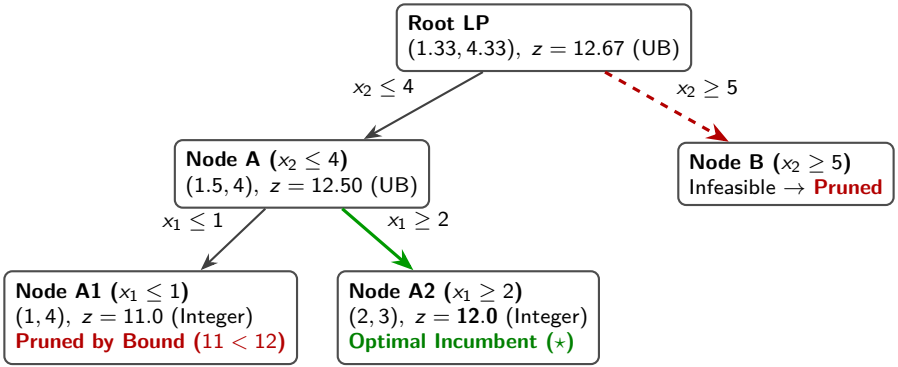
$$2x_1 + x_2 \leq 7,$$
$$-x_1 + x_2 \leq 3,$$
$$x_1, x_2 \geq 0, \quad x_1, x_2 \in \mathbb{Z}.$$

Root LP Relaxation Optimum:
$(x_1^*, x_2^*) = \left(\frac{4}{3}, \frac{13}{3}\right), \quad z^* \approx 12.67$ (fractional).

B&B Step 1: Branch on the Fractional Variable



B&B Step 2: Complete Search Tree and Bounds



Section 2

The 0-1 Knapsack Problem

Problem Definition: The 0-1 Knapsack

What is the 0-1 Knapsack Problem?

The Classic Analogy

Imagine you have a knapsack (or bag) with a limited weight capacity, W.

- You are presented with n different items.
- Each item i has:
  - A **value**, v_i
  - A **weight**, w_i
- You must make a binary ("0-1") decision for each item:
  - Either you **take** the item ($x_i = 1$)
  - Or you **leave** the item ($x_i = 0$)
- You cannot take fractions of items.

The Goal

Choose the combination of items that **maximizes the total value** in your knapsack...

...without **exceeding the total weight capacity** W.

A Simple Knapsack Example

Which Items Should We Take?

Let's define a specific problem:

Knapsack Capacity

Maximum Weight (W) = **10 kg**

Available Items ($n = 4$)

Item (i)	**Name**	**Value (v_i)**	**Weight (w_i)**
1	Laptop	\$40	4 kg
2	Gold	\$50	3 kg
3	Silver	\$30	6 kg
4	Books	\$10	5 kg

The Question

How do we formally model the decision to maximize total value without exceeding 10 kg?

ILP Formulation

Translating the Problem into Math

We can model this problem using Binary Integer Linear Programming:

1. Decision Variables

- For each item i, define a binary variable $x_i \in \{0, 1\}$:
- $x_i = 1$ if we **take** item i
- $x_i = 0$ if we **leave** item i

x_1: Laptop, x_2: Gold, x_3: Silver, x_4: Books

2. Objective Function (Maximize Value)

Maximize the total value of selected items:

$$\max z = \sum_{i=1}^{4} v_i x_i = 40x_1 + 50x_2 + 30x_3 + 10x_4$$

3. Constraint (Weight Capacity)

Total weight cannot exceed $W = 10$:

$$4x_1 + 3x_2 + 6x_3 + 5x_4 \leq 10$$

How Does a "Greedy" Heuristic Work?

- Rank items by highest "bang for the buck" (value per kg, v_i/w_i):
  - **1. Gold:** \$50/3 kg = \$16.67/kg
  - **2. Laptop:** \$40/4 kg = \$10.00/kg
  - **3. Silver:** \$30/6 kg = \$5.00/kg
  - **4. Books:** \$10/5 kg = \$2.00/kg
- **Greedy Step 1:** Take Gold ($x_2 = 1$). *Value = \$50, Weight = 3 kg (7 kg left).*
- **Greedy Step 2:** Take Laptop ($x_1 = 1$). *Value* = \$50 + \$40 = **\$90**, *Weight* $= 3 + 4 = 7$ *kg (3 kg left).*
- Neither Silver (6 kg) nor Books (5 kg) fit in the remaining 3 kg.
- **Greedy Candidate:** {Laptop, Gold} with $z =$ \$90.

The Complete 0-1 Knapsack ILP Model

$$\begin{aligned} \max \quad & z = 40x_1 + 50x_2 + 30x_3 + 10x_4 \\ \text{s.t.} \quad & 4x_1 + 3x_2 + 6x_3 + 5x_4 \leq 10 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

Optimal ILP Solution

- Solving the exact ILP confirms that the greedy candidate **is** optimal in this instance:
- $x_1^* = 1,\ x_2^* = 1,\ x_3^* = 0,\ x_4^* = 0$
- Optimal Value: $z^* = 40(1) + 50(1) + 0 + 0 = \mathbf{\$90}$
- Total Weight: $w = 4(1) + 3(1) = 7 \text{ kg} \leq 10 \text{ kg}$
- *Comparison:* {Gold, Silver} uses more capacity ($w = 9$ kg) but yields lower value ($z = \$80$).

Reflecting on the Knapsack Solution

- Greedy heuristics often **fail** for 0-1 Knapsack problems (even though it happened to match the optimum here).
- *Counterexample:* If Silver weighed 7 kg with value \$45 ($v_3/w_3 = \$6.43/\text{kg}$), Greedy would still pick {Gold, Laptop} ($z = \$90, w = 7$), whereas {Gold, Silver} would achieve $z = \$95, w = 10$!
- The ILP formulation guarantees the true global optimum.
- For large n, 0-1 Knapsack is $\mathcal{NP}$-hard and solved via Branch-and-Bound or Dynamic Programming.

Solving the 0-1 Knapsack in Python (PuLP 4.0)

Example 2: Binary Decision Variables with `add_variable_dicts`

```python
import pulp

items = ["Laptop", "Gold", "Silver", "Books"]
val = {"Laptop": 40, "Gold": 50, "Silver": 30, "Books": 10}
wt  = {"Laptop": 4,  "Gold": 3,  "Silver": 6,  "Books": 5}

prob = pulp.LpProblem("Knapsack_0_1", pulp.LpMaximize)

# Binary variables x[i] in {0, 1} using PuLP 4.0 API
x = prob.add_variable_dicts("x", items, cat=pulp.LpBinary)

# Objective & Weight Capacity Constraint
prob += pulp.lpSum(val[i] * x[i] for i in items), "Total_Value"
prob += pulp.lpSum(wt[i] * x[i] for i in items) <= 10, "Max_Weight"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Optimal Value = ${z_opt:.0f}")
for i in items:
    if x[i].varValue > 0.5:
        print(f"  Take {i}: weight={wt[i]} kg, value=${val[i]}")
```

Console Output

```
Status:  Optimal
Optimal Value = $90
Take Laptop:  weight=4 kg,
value=$40
Take Gold:  weight=3 kg, value=$50
```

Key Takeaway

`cat=pulp.LpBinary` restricts each variable to $\{0, 1\}$. Using `prob.add_variable_dicts` scales cleanly to hundreds of items.

Section 3

The 0-1 Knapsack Problem: Logical Constraints

The Base Knapsack Problem (KP)

Standard Binary Integer Programming Model

Decision Variables

For each item i, let x_i be a **binary decision variable**:

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

Model Notation

- v_i: Value of item i
- w_i: Weight of item i
- C: Capacity of the knapsack

The Standard KP Model

$$\max \sum_{i \in I} v_i x_i$$

$$\text{s.t.} \quad \sum_{i \in I} w_i x_i \leq C$$

$$x_i \in \{0, 1\}, \quad \forall i \in I$$

- Next, we add **logical conditions** (implications, requirements, conflicts) using linear inequalities.

Complication 1: Conditional Inclusion

Rule: If Item 1 is Taken, Then Item 2 Must Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 1$

This is a logical implication constraint. If the premise ($x_1 = 1$) is true, the conclusion ($x_2 = 1$) must also be true.

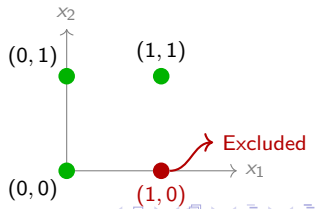
Test Cases (All Combinations)

- $x_1 = 1, x_2 = 0 \implies 1 - 0 = 1 \not\leq 0$ (**Violates** → Excluded)
- $x_1 = 1, x_2 = 1 \implies 1 - 1 = 0 \leq 0$ (**Satisfies** → Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 - 0 = 0 \leq 0$ (**Satisfies** → Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 - 1 = -1 \leq 0$ (**Satisfies** → Feasible)

The MILP Constraint

$$\mathbf{x_1 - x_2 \leq 0} \quad (\text{or } \mathbf{x_1 \leq x_2})$$

Visualization of Feasible Combinations



Complication 2: Mandatory Inclusion

Rule: Item 1 Must Be Taken

Constraint Rule: $x_1 = 1$

This is the simplest form of logical constraint, forcing a specific binary decision to be true regardless of other factors (as long as capacity allows).

Impact on the Model

- All feasible solutions must now include Item 1.
- The objective and capacity constraints effectively become:

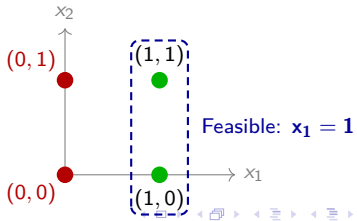
$$\max \left(v_1 + \sum_{i \in I \setminus \{1\}} v_i x_i \right)$$

$$\text{s.t.} \quad w_1 + \sum_{i \in I \setminus \{1\}} w_i x_i \leq C$$

The MILP Constraint

$$\mathbf{x_1 = 1}$$

Visualization: Forcing $x_1 = 1$



Complication 3: Mutual Exclusion (Prohibition)

Rule: If Item 1 is Taken, Then Item 2 Cannot Be Taken

Constraint Rule: $x_1 = 1 \implies x_2 = 0$

This relationship is symmetric for binary variables. If we take Item 1, we exclude Item 2, and if we take Item 2, we exclude Item 1. This is known as a **Mutual Exclusion** constraint.

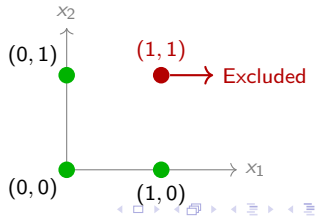
Test Cases (All Combinations)

- $x_1 = 1, x_2 = 1 \implies 1 + 1 = 2 \not\leq 1$ (**Violates** $\rightarrow$ Excluded)
- $x_1 = 1, x_2 = 0 \implies 1 + 0 = 1 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 + 1 = 1 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 + 0 = 0 \leq 1$ (**Satisfies** $\rightarrow$ Feasible)

The MILP Constraint

$$\mathbf{x_1 + x_2 \leq 1}$$

Visualization of Feasible Combinations



Complication 4: Exclusive OR (XOR)

Rule: Either Item 1 OR Item 2 Must Be Taken (But Not Both)

Constraint Rule: ($x_1 = 1$ XOR $x_2 = 1$)

This constraint requires an exact choice of one item from the pair: **at least one** must be chosen ($x_1 + x_2 \geq 1$), and **at most one** can be chosen ($x_1 + x_2 \leq 1$).

Test Cases (All Combinations)

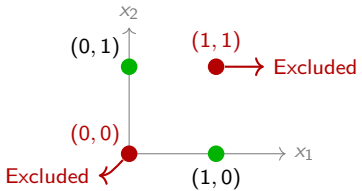
- $x_1 = 1, x_2 = 1 \implies 1 + 1 = 2 \neq 1$ (Excluded)
- $x_1 = 1, x_2 = 0 \implies 1 + 0 = 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 1 \implies 0 + 1 = 1$ (**Satisfies** $\rightarrow$ Feasible)
- $x_1 = 0, x_2 = 0 \implies 0 + 0 = 0 \neq 1$ (Excluded)

- Only the diagonal combinations $(\mathbf{1}, \mathbf{0})$ and $(\mathbf{0}, \mathbf{1})$ are allowed.

The MILP Constraint

$$\mathbf{x_1 + x_2 = 1}$$

Visualization of Feasible Combinations



Knapsack with Logical Constraints in Python (PuLP 4.0)

Example 3: Adding Mutual Exclusion and Implication Rules

```python
import pulp

items = ["Laptop", "Gold", "Silver", "Books"]
val = {"Laptop": 40, "Gold": 50, "Silver": 30, "Books": 10}
wt  = {"Laptop": 4,  "Gold": 3,  "Silver": 6,  "Books": 5}

prob = pulp.LpProblem("Knapsack_Logical", pulp.LpMaximize)
x = prob.add_variable_dicts("x", items, cat=pulp.LpBinary)

prob += pulp.lpSum(val[i] * x[i] for i in items), "Total_Value"
prob += pulp.lpSum(wt[i] * x[i] for i in items) <= 10, "Max_Weight"

# Rule 1 (Mutual Exclusion): Cannot take both Laptop and Gold
prob += x["Laptop"] + x["Gold"] <= 1, "Mutually_Exclusive"

# Rule 2 (Implication): If Gold is taken, Books must be taken
prob += x["Gold"] - x["Books"] <= 0, "Gold_Implies_Books"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Optimal Value = ${z_opt:.0f}")
print("Selected:", [i for i in items if x[i].varValue > 0.5])
```

Impact of Logical Rules

- **Option A (Take Gold):** Forces Books ($3 + 5 = 8$ kg), blocks Laptop; remaining 2 kg fits nothing
  $\Rightarrow z = 50 + 10 = \$60$.
- **Option B (Take Laptop + Silver):**
  $4 + 6 = 10$ kg ≤ 10 kg
  $\Rightarrow z = 40 + 30 = \mathbf{\$70}$.

Console Output

```
Status:  Optimal, Optimal Value = $70
Selected:  ['Laptop', 'Silver']
```

Section 4

Modeling Fixed Setup Costs

Problem Scenario: Capacity and Production

The Fixed Capacity Decision

A company manufactures two products, x_1 and x_2. They earn profits of **\$5 per unit of** x_1 and **\$3 per unit of** x_2.

The Fixed Cost Element

Before any production can occur (i.e., if $x_1 > 0$ or $x_2 > 0$), a single facility must be activated at a **fixed cost of \$10**.

Constraints (Resources)

- $\mathbf{C_1}$ (Resource A): $x_1 + x_2 \leq 10$
- $\mathbf{C_2}$ (Resource B): $2x_1 + x_2 \leq 16$
- $\mathbf{C_3}$ (Resource C): $x_2 \leq 7$

Decision Required

The final solution must choose between:

1. **No Production ($y = 0$):** Total Profit $Z = 0$.
2. **Production ($y = 1$):** Total Profit = (Gross Profit) − \$10.

Step 1: The Continuous LP Relaxation

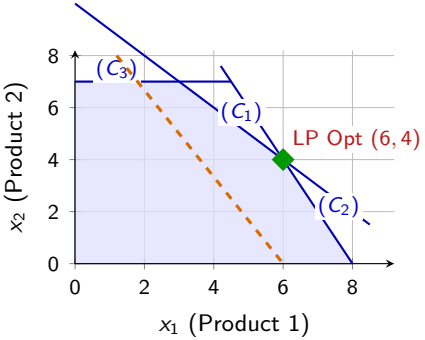
Solving for Maximum Gross Profit

We first maximize the gross profit $\mathbf{Z_{gross} = 5x_1 + 3x_2}$ while ignoring the fixed cost of \$10.

LP Formulation

$$\max Z_{\text{gross}} = 5x_1 + 3x_2$$

$$\begin{aligned} \text{s.t.} \quad & x_1 + x_2 \leq 10 && (C_1) \\ & 2x_1 + x_2 \leq 16 && (C_2) \\ & x_2 \leq 7 && (C_3) \\ & x_1, x_2 \geq 0 \end{aligned}$$



Result of LP Relaxation (Gross Profit)

The Gross Optimal Solution

- The optimal solution to the continuous Linear Program is found at the intersection of constraints $\mathbf{C_1}$ and $\mathbf{C_2}$:

$$\mathbf{C_1}: x_1 + x_2 = 10 \quad \Big| \quad \mathbf{C_2}: 2x_1 + x_2 = 16$$

- Solving these equations yields:

$$x_1^* = 6, \quad x_2^* = 4$$

- **Maximum Gross Profit (Z_{gross}):**

$$Z_{\text{gross}} = 5(6) + 3(4) = 30 + 12 = \$\mathbf{42}$$

If we use this LP solution, what is the NET Profit?

$$Z_{\text{net}} = Z_{\text{gross}} - \text{Fixed Cost} = 42 - 10 = \$\mathbf{32}$$

This establishes the maximum net profit if we decide to activate the facility.

Step 2: The Full MILP Formulation

Introducing the Binary Variable

We introduce a **binary variable** $y \in \{0, 1\}$ to enforce the fixed cost and link production levels to the activation decision.

Decision Variables

- $\mathbf{x_1, x_2} \in \mathbb{R}_{\geq \mathbf{0}}$ (Production quantities)
- $\mathbf{y} \in \{\mathbf{0}, \mathbf{1}\}$ (Facility activation indicator)

Objective Function (Net Profit)

The objective now includes the fixed cost, dependent on y:

$$\max Z = 5x_1 + 3x_2 - 10y$$

Fixed Cost Linking Constraint (Big-M)

We must link x_1, x_2 to y using a **Big-M** constraint:

- If $y = 0$, then $x_1 = x_2 = 0$.
- If $y = 1$, then x_1, x_2 are free to take feasible values.

Let $M = 10$ (tight upper bound on $x_1 + x_2$ from C_1):

$$\mathbf{x_1 + x_2 \leq 10y} \quad \Longleftrightarrow \quad \mathbf{x_1 + x_2 - 10y \leq 0}$$

The Complete Mixed-Integer Linear Program (MILP):

$$\max Z = 5x_1 + 3x_2 - 10y$$

$$\text{s.t.} \begin{cases} x_1 + x_2 \leq 10 & (C_1 : \text{Resource A}) \\ 2x_1 + x_2 \leq 16 & (C_2 : \text{Resource B}) \\ x_2 \leq 7 & (C_3 : \text{Resource C}) \\ \mathbf{x_1 + x_2 - 10y \leq 0} & (\text{Big-}M \text{ Linking Constraint}) \\ x_1, x_2 \geq 0, \quad \mathbf{y} \in \{\mathbf{0}, \mathbf{1}\} & \end{cases}$$

Step 3: Solving the MILP – Case 1

Case 1: Facility Not Activated ($y = 0$)

If the company chooses **not** to pay the fixed cost, then $y = 0$.

- **Fixed-cost constraint becomes:**

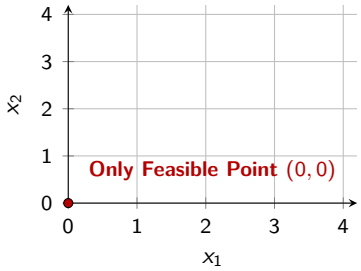
$$x_1 + x_2 - 10(0) \leq 0 \implies x_1 + x_2 \leq 0$$

- Since $x_1 \geq 0$ and $x_2 \geq 0$, the only feasible solution is:

$$\mathbf{x_1 = 0},\ \mathbf{x_2 = 0}.$$

- **Total profit:**

$$Z = 5(0) + 3(0) - 10(0) = \$\mathbf{0}.$$



Step 4: Solving the MILP – Case 2

Case 2: Facility Activated ($y = 1$)

If the company **pays** the fixed cost ($y = 1$), we use the LP optimal found in Step 1.

- **Fixed Cost Constraint becomes:**

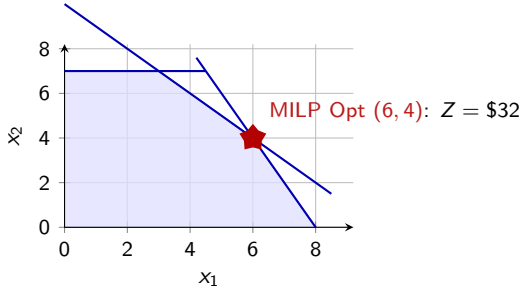
$$x_1 + x_2 - 10(1) \leq 0 \quad \Longrightarrow \quad x_1 + x_2 \leq 10$$

- This is identical to C_1, so the feasible region remains the original shaded polygon.
- The optimal production mix is the LP solution:

$$\mathbf{x_1^* = 6}, \quad \mathbf{x_2^* = 4}$$

- **Total Net Profit (Z):**

$$Z = 5(6) + 3(4) - 10(1) = 42 - 10 = \mathbf{\$32}$$



Conclusion: The Optimal MILP Solution

Comparing the Binary Choices

- **Case 1 (No Capacity):** $y = 0 \implies Z = \$0$
- **Case 2 (Capacity Activated):** $y = 1 \implies Z = \$32$

Optimal Decision

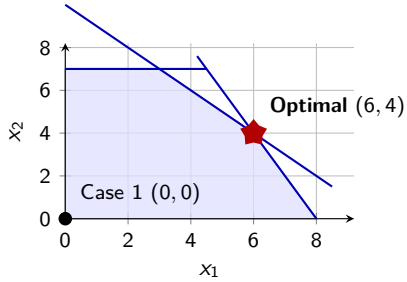
The optimal profit is $\mathbf{Z^* = \$32}$, achieved in Case 2.

Optimal MILP Solution

- $\mathbf{y^* = 1}$ (Activate facility)
- $\mathbf{x_1^* = 6}$ units, $\mathbf{x_2^* = 4}$ units

The fixed cost ($10) is outweighed by the gross profit ($42).

Visualization of the Two Candidate Optima



Fixed Setup Cost MILP in Python (PuLP 4.0)

Example 4: Modeling Big-M Linking Constraints

```
import pulp

prob = pulp.LpProblem("Fixed_Setup_Cost_MILP", pulp.LpMaximize)

# Continuous production variables and binary setup variable
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpContinuous)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpContinuous)
y  = prob.add_variable("y", cat=pulp.LpBinary)

# Objective: Gross Profit - Fixed Setup Cost
prob += 5 * x1 + 3 * x2 - 10 * y, "Net_Profit"

# Resource constraints + Big-M linking constraint (M = 10)
prob += x1 + x2 <= 10, "Resource_A"
prob += 2 * x1 + x2 <= 16, "Resource_B"
prob += x2 <= 7, "Resource_C"
prob += x1 + x2 - 10 * y <= 0, "Big_M_Link"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Net Profit Z* = ${z_opt:.2f}")
print(f"Activate y*={int(y.varValue)}, x1*={x1.varValue}, x2*={x2.varValue}")
```

Console Output

```
Status:  Optimal
Net Profit Z* = $32.00
Activate Facility y* = 1
x1* = 6.0, x2* = 4.0
```

Why Big-M Matters

Without `x1 + x2 - 10 * y <= 0`, the solver would set `y = 0` to avoid the \$10 fee while still producing `x1 = 6, x2 = 4`!

Section 5

Modeling a Min Purchase Quantity

Problem Description: Minimum Purchase Quantity

Context:

- A company can order two products — A and B — from a single supplier.
- Ordering from the supplier incurs a fixed setup (shipping) cost.
- If an order is placed, the supplier requires a **minimum purchase quantity (MOQ)** of 6 total units.
- Each product consumes limited assembly hours and materials.
- The firm must decide how many units of each product to order, and whether to place an order at all.

Goal:

- Maximize total profit = revenue from A and B − fixed ordering cost.

Decision Variables and Parameters

Decision Variables:

- x_1 = quantity of Product A to order (units)
- x_2 = quantity of Product B to order (units)
- $y = \begin{cases} 1 & \text{if an order is placed,} \\ 0 & \text{otherwise.} \end{cases}$

Parameters:

- Profit per unit: \$7 for A, \$5 for B
- Fixed ordering cost: \$12
- Assembly time: 2 hours for A, 1 hour for B (max 16 hours)
- Material usage: 1 for A, 2 for B (max 18 units)
- Demand limit for B: $x_2 \leq 8$
- Minimum purchase quantity: at least 6 units total if an order is placed

MILP with Minimum Purchase Quantity (MOQ)

Supplier Order with MOQ:

- Two products (A and B) assembled from the same supplier.
- If we place an order ($y = 1$), we pay a fixed shipping/setup cost of \$12 and must buy at least **6** total units.

$$\max\ Z = 7x_1 + 5x_2 - 12y$$

$$\text{s.t.} \begin{cases} \text{Assembly hours:} & 2x_1 + x_2 \le 16 \\ \text{Material limit:} & x_1 + 2x_2 \le 18 \\ \text{Demand cap for B:} & x_2 \le 8 \\ \textbf{MOQ if ordering:} & \mathbf{x_1 + x_2 \ge 6\,y} \\ \textbf{No purchase unless ordering:} & \mathbf{x_1 \le 18\,y},\ \mathbf{x_2 \le 18\,y} \\ x_1, x_2 \ge 0, \quad \mathbf{y \in \{0,1\}} \end{cases}$$

Solving the MOQ Model: Optimal Solution & Analysis

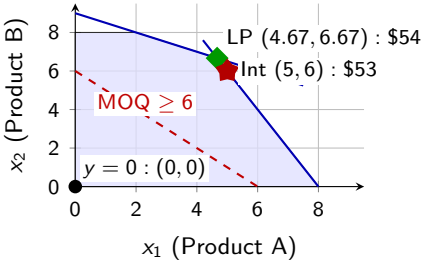
Comparing No-Order ($y = 0$) vs. Active Order ($y = 1$)

Case 1: Do Not Order ($y = 0$)

- Upper bounds force $x_1 = 0,\ x_2 = 0 \implies Z = \$\mathbf{0}$.

Case 2: Place Order ($y = 1$)

- Must satisfy $x_1 + x_2 \geq 6$ plus resource limits.
- **Continuous** $x_1, x_2 \geq 0$**:** Intersection of Assembly ($2x_1 + x_2 = 16$) and Material ($x_1 + 2x_2 = 18$): $x_1^* = \frac{14}{3} \approx 4.67,\ \ x_2^* = \frac{20}{3} \approx 6.67,\ \ Z^* = \$\mathbf{54.00}$.
- **Integer** $x_1, x_2 \in \mathbb{Z}_{\geq 0}$**:** $x_1^* = 5,\ \ x_2^* = 6 \implies Z^* = 7(5) + 5(6) - 12 = \$\mathbf{53.00}$.



Solving the MOQ Model in Python (PuLP 4.0)

Example 5: Disjunctive Feasible Region with Minimum Order Quantity

```python
import pulp

prob = pulp.LpProblem("Supplier_Order_MOQ", pulp.LpMaximize)

# Integer order quantities x1, x2 and binary order indicator y
x1 = prob.add_variable("x1", lowBound=0, cat=pulp.LpInteger)
x2 = prob.add_variable("x2", lowBound=0, cat=pulp.LpInteger)
y  = prob.add_variable("y", cat=pulp.LpBinary)

prob += 7 * x1 + 5 * x2 - 12 * y, "Net_Profit"

prob += 2 * x1 + x2 <= 16, "Assembly_Hours"
prob += x1 + 2 * x2 <= 18, "Material_Limit"
prob += x2 <= 8, "Demand_Cap_B"
prob += x1 + x2 - 6 * y >= 0, "MOQ_Constraint"
prob += x1 - 18 * y <= 0, "Link_x1"
prob += x2 - 18 * y <= 0, "Link_x2"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Profit Z* = ${z_opt:.2f}")
print(f"y*={y.varValue:.0f}, x1*={x1.varValue:.0f}, x2*={x2.varValue:.0f}")
```

Console Output

```
Status:  Optimal
Profit Z* = $53.00
Order y* = 1
x1* = 5, x2* = 6
```

Modeling Insight

Combining $\mathbf{x_1 + x_2 \geq 6y}$ with $\mathbf{x_i \leq 18y}$ creates a *semi-continuous* region: total quantity is either exactly 0 ($y = 0$) or at least 6 ($y = 1$).

Conclusion

- **Computational Complexity:** ILPs are $\mathcal{NP}$-hard in general; solving an ILP requires solving a sequence of LP relaxations via Branch-and-Bound and Cutting Planes.
- **Solver Ecosystem:** Industrial-scale commercial MILP solvers (e.g., Gurobi, CPLEX) are expensive, whereas open-source MILP solvers like **CBC** (bundled free with Python's `PuLP`) solve small-to-medium ILPs effectively.
- **Duality & Sensitivity:** Standard LP shadow prices and reduced costs do **not** directly apply to ILPs because discrete jumps break continuity.
- **Core Applications:** 0-1 selection problems (knapsack, assignment), logical conditions, fixed setup costs, and minimum order quantities (MOQ).

Section 6

Comments about the Upcoming Midterm

Exam Structure

- **Online exam:** between 35 and 40 questions
- Availability and submission deadlines are on Canvas
- Exam is not timed
- Exam can be paused and resumed
- Two attempts available (grade based on the **last** attempt)
- If the average grade is less than 85%, the exam will be curved to an average of 85%
- Exam is open book, notes, etc.
- Okay to use AI
- Exam is an individual effort

Topics Covered on the Midterm

- Basic Python trivia questions
- Basic Spreadsheet trivia
- **Linear programming**
  - product mix
  - staffing
  - transportation
  - assignment
  - blending
- **Sensitivity analysis**
  - Reduced Costs
  - Shadow Prices
- **Integer linear programming**
  - 0-1 problems
  - fixed cost problems

Resources

- Practice problems posted on Canvas
- Homeworks!
- AI: utilize what is available, go crazy.

Section 7

Practice Problems

Staffing Problem: Description

Problem Context:

- A call center must schedule workers to cover daily shifts.
- Each shift requires a minimum number of employees.
- Employees work 5 consecutive days followed by 2 days off.
- The objective is to minimize total staffing costs while meeting coverage.

Goal:

- Decide how many employees to start on each day of the week.
- Ensure daily staffing requirements are satisfied at minimum cost.

Staffing Problem: Decision Variables and Constraints

Decision Variables:

- x_i = number of employees who start their 5-day workweek on day i, $i = 1, \ldots, 7$

Parameters:

- c_i = cost of assigning an employee starting on day i
- r_j = minimum required employees on day j

Key Constraints:

- Each day must have at least r_j employees working.
- Employees work 5 consecutive days, off 2 days.

Staffing Problem: LP Formulation

$$\min Z = \sum_{i=1}^{7} c_i x_i$$

$$\text{s.t.} \begin{cases} x_1 + x_4 + x_5 + x_6 + x_7 \geq r_1 & \text{(Mon)} \\ x_1 + x_2 + x_5 + x_6 + x_7 \geq r_2 & \text{(Tue)} \\ x_1 + x_2 + x_3 + x_6 + x_7 \geq r_3 & \text{(Wed)} \\ x_1 + x_2 + x_3 + x_4 + x_7 \geq r_4 & \text{(Thu)} \\ x_1 + x_2 + x_3 + x_4 + x_5 \geq r_5 & \text{(Fri)} \\ x_2 + x_3 + x_4 + x_5 + x_6 \geq r_6 & \text{(Sat)} \\ x_3 + x_4 + x_5 + x_6 + x_7 \geq r_7 & \text{(Sun)} \\ x_i \geq 0 \quad \forall i \end{cases}$$

Blending Problem: Description

Problem Context:

- A refinery produces a fuel blend using two raw materials.
- Each raw material has a different cost and quality content.
- The final product must meet a minimum quality standard.
- The goal is to minimize total cost while satisfying demand and quality.

Blending Problem: Decision Variables and Constraints

Decision Variables:

- x_1 = barrels of crude A used
- x_2 = barrels of crude B used

Parameters:

- c_1, c_2 = costs per barrel of A and B
- q_1, q_2 = quality measures of A and B
- $Q_{\min}$ = minimum required blend quality
- D = demand (barrels of finished blend)

Key Constraints:

- $x_1 + x_2 = D$ (total demand)
- Weighted average quality $\geq Q_{\min}$
- $x_1, x_2 \geq 0$

Blending Problem: LP Formulation

$$\min Z = c_1 x_1 + c_2 x_2$$

$$\text{s.t.} \begin{cases} x_1 + x_2 = D \\ q_1 x_1 + q_2 x_2 \geq Q_{\min}(x_1 + x_2) \\ x_1, x_2 \geq 0 \end{cases}$$

Interpretation: Choose crude mix to meet quality and demand at lowest cost.

Transshipment Problem: Description

Problem Context:

- A company ships goods from factories to warehouses, then to retailers.
- Each arc (route) has a cost per unit shipped.
- Factory supplies, warehouse capacities, and retailer demands are known.
- The objective is to minimize total shipping cost.

Transshipment Problem: Decision Variables and Constraints

Decision Variables:

- x_{ij} = units shipped from node i to node j

Parameters:

- c_{ij} = shipping cost per unit from i to j
- s_i = supply at source i
- d_j = demand at destination j

Key Constraints:

- Flow conservation at each node:

$$\sum_j x_{ij} - \sum_k x_{ki} = b_i$$

where $b_i > 0$ for sources, $b_i < 0$ for sinks, $b_i = 0$ for transshipment nodes.

Transshipment Problem: LP Formulation

$$\min Z = \sum_i \sum_j c_{ij} x_{ij}$$

$$\text{s.t.} \quad \sum_j x_{ij} - \sum_k x_{ki} = b_i \quad \forall i$$

$$x_{ij} \geq 0 \quad \forall i, j$$

Interpretation: Ship goods through a network of factories, warehouses, and retailers to minimize total transport cost.

Assignment Problem: Description

Problem Context:

- A manager must assign a set of workers to a set of tasks.
- Each worker can perform only one task, and each task requires exactly one worker.
- The cost (or time) of assigning worker i to task j is known.
- The objective is to minimize the total assignment cost.

Goal:

- Determine the optimal one-to-one assignment between workers and tasks that yields the lowest total cost.

Assignment Problem: Decision Variables and Constraints

Decision Variables:

- $x_{ij} = \begin{cases} 1 & \text{if worker } i \text{ is assigned to task } j \\ 0 & \text{otherwise} \end{cases}$

Parameters:

- c_{ij} = cost (or time) of assigning worker i to task j

Key Constraints:

- Each worker is assigned to exactly one task:

$$\sum_j x_{ij} = 1 \quad \forall i$$

- Each task receives exactly one worker:

$$\sum_i x_{ij} = 1 \quad \forall j$$

- Binary restriction: $x_{ij} \in \{0, 1\}$

Assignment Problem: Full LP Formulation

$$\min Z = \sum_i \sum_j c_{ij} x_{ij}$$

$$\text{s.t.} \begin{cases} \sum_j x_{ij} = 1 & \forall i \quad \text{(each worker does one task)} \\ \sum_i x_{ij} = 1 & \forall j \quad \text{(each task has one worker)} \\ x_{ij} \in \{0,1\} & \forall i,j \end{cases}$$

Interpretation: Find the assignment pattern minimizing total cost (or maximizing efficiency) under one-to-one matching constraints.

New Product Decision: Problem Description

Context:

- A firm produces Products A and B using two scarce resources: Machine Hours (M) and Labor Hours (L).
- A new Product C is being considered.
- We can use **shadow prices** and **reduced cost** from the current LP solution to decide if C should be added.

Goal:

- Determine whether introducing Product C would increase total profit.

Base Linear Program (Current Products A and B)

Current LP:

$$\max Z = 30x_A + 20x_B$$

$$\text{s.t.} \begin{cases} 2x_A + 1x_B \leq 100 & \text{(Machine hours M)} \\ 1x_A + 1.5x_B \leq 90 & \text{(Labor hours L)} \\ x_A, x_B \geq 0 \end{cases}$$

Optimal Primal Solution:

$$x_A^\star = 30, \quad x_B^\star = 40, \quad Z^\star = 30(30) + 20(40) = \$\mathbf{1700}$$

Unique Shadow Prices (from binding constraints M and L):

$$\begin{cases} 2y_M^\star + 1y_L^\star = 30 \\ 1y_M^\star + 1.5y_L^\star = 20 \end{cases} \implies \mathbf{y_M^\star = \$12.50}, \quad \mathbf{y_L^\star = \$5.00}$$

Check Dual Objective: $100(12.50) + 90(5.00) = 1250 + 450 = \$1700 = Z^\star$

Candidate Product C

Product C Data:

	Profit c_i	Resource Use (M, L)
Product C	\$26	(1, 2)

Idea:

- Product C earns \$26 per unit and uses 1 hour of Machine time and 2 hours of Labor.
- We test profitability using the *reduced cost* (net marginal benefit) formula:

$$r_C = c_C - \sum_i y_i^\star a_{iC} = c_C - (y_M^\star a_{MC} + y_L^\star a_{LC})$$

Reduced Cost Calculation and Decision

Compute Reduced Cost for Product C:

$$r_C = 26 - (12.50 \times 1 + 5.00 \times 2) = 26 - (12.50 + 10.00) = \boxed{+\$3.50 > 0}$$

Interpretation:

- $r_C = +\$3.50 > 0 \implies$ Each unit of Product C adds \$3.50 in net profit above the opportunity cost of its resources (\$22.50).
- Therefore, **Product C should be produced** (it enters the optimal basis).

Example of a Non-Profitable Candidate (Product D, $c_D = \$18$, uses 2 M, 1 L):

$$r_D = 18 - (12.50 \times 2 + 5.00 \times 1) = 18 - 30.00 = -\$12.00 \leq 0$$

Product D should not be added (opportunity cost of resources is \$30, exceeding its \$18 profit).

Interpreting Shadow Prices

Shadow Price ($y_i^\star$):

- Represents the change in optimal objective value $Z^\star$ per unit increase in the RHS of a binding constraint.
- Example (Machine Hours):

$$\frac{\Delta Z^\star}{\Delta \text{Machine hours}} = y_M^\star = \$12.50$$

- Meaning: one extra machine hour increases maximum profit by **$12.50**.

Valid Range:

- Shadow prices remain constant only within the allowable RHS range where the current optimal basis remains feasible ($60 \leq \text{RHS}_M \leq 180$).

Effect of RHS (Resource) Changes

Scenario: Increase Machine Hours by 5 units ($\mathbf{RHS}_M : 100 \to 105$)

$$\Delta Z = y_M^\star \times 5 = 12.50 \times 5 = \$62.50$$

$$Z_{\text{new}} = 1700 + 62.50 = \boxed{\$1762.50}$$

Interpretation:

- Within the allowable range, each additional 1 hour of M raises profit by \$12.50.
- If M increases beyond 180 hours, Labor becomes the sole bottleneck, the basis changes, and $y_M^\star = \$12.50$ no longer holds.

Takeaway: Shadow prices and reduced costs help managers evaluate new products or resource changes *without re-solving the LP from scratch*.

Verifying Shadow Prices & Reduced Costs in Python (PuLP 4.0)

Example 6: Extracting Dual Prices (c.pi) and Testing Candidate Products

```
import pulp

prob = pulp.LpProblem("Base_Product_Mix", pulp.LpMaximize)
xA = prob.add_variable("xA", lowBound=0, cat=pulp.LpContinuous)
xB = prob.add_variable("xB", lowBound=0, cat=pulp.LpContinuous)

prob += 30 * xA + 20 * xB, "Total_Profit"
prob += 2 * xA + 1 * xB <= 100, "Machine_Hours"
prob += 1 * xA + 1.5 * xB <= 90, "Labor_Hours"

res = prob.solve()
z_opt = pulp.value(prob.objective)
print(f"Status: {res.status.name}, Z* = ${z_opt:.2f}")
print(f"xA* = {xA.varValue:.1f}, xB* = {xB.varValue:.1f}")

# Extract Shadow Prices from prob.constraints() list in PuLP 4.0
duals = {c.name: abs(c.pi) for c in prob.constraints()}
print("Shadow Prices:", duals)
r_C = 26 - (1 * duals["Machine_Hours"] + 2 * duals["Labor_Hours"])
print(f"Reduced Cost of Product C: r_C = +${r_C:.2f}")
```

Console Output

```
Status:  Optimal, Z* = $1700.00
xA* = 30.0, xB* = 40.0
Shadow Prices:
{'Machine_Hours':  12.5,
 'Labor_Hours':  5.0}
Reduced Cost of Product C:
r_C = +$3.50
```

PuLP 4.0 Note

In PuLP 4.0, `prob.constraints()` is a method returning a list of constraint objects. Access `c.name` and `abs(c.pi)` to retrieve shadow prices!